

User Manual

for S32K1_S32M24X DPGA Driver

Document Number: UM2DPGAASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	3
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.3 Hardware Resources	6
3.4 Deviations from Requirements	6
3.5 Driver Limitations	7
3.6 Driver usage and configuration tips	7
3.6.1 Dpga introduction	7
3.6.2 Dpga Amplifier	7
3.6.3 Dpga Blanking Time	8
3.7 Runtime errors	8
3.8 Symbolic Names Disclaimer	9
3.9 Usage in non AUTOSAR context	9
4 Configuration Parameters	12
4.1 Dpga	13
4.1.1 Module Dpga	14
4.1.2 Container GeneralConfiguration	14
4.1.3 Parameter DpgaEnableVoltageMonitoring	14
4.1.4 Parameter DpgaDevErrorDetect	15
4.1.5 Parameter DpgaIpDevErrorDetect	15
4.1.6 Parameter DpgaMultiPartitionSupport	16
4.1.7 Parameter DpgaEnableUserModeSupport	16
4.1.8 Parameter DpgaVersionInfoApi	16
4.1.9 Parameter DpgaVoltageMonitorSelfTestApi	17
4.1.10 Parameter DpgaAmplifierSelfTestApi	17
4.1.11 Parameter DpgaAmplifierSelfTestTolerance	18
4.1.12 Parameter DpgaTimeoutMethod	18
4.1.13 Parameter DpgaTimeoutValue	18
4.1.14 Reference DpgaEcucPartitionRef	19
4.1.15 Reference DpgaAmplifierSelfTestAdcReference	19
4.1.16 Container DpgaChannel	20
4.1.17 Parameter DpgaChannelId	20
4.1.18 Parameter DpgaHwChannel	20
4.1.19 Container DpgaConfiguration	21
4.1.20 Parameter DpgaAmplifierGain	21
4.1.21 Parameter DpgaOutCommonModeVoltage	22
4.1.22 Parameter DpgaOffset	22
4.1.23 Parameter DpgaInCommonModeCoarse	23
4.1.24 Parameter DpgaInCommonModeFine	24
4.1.25 Parameter DpgaBlankingTimeDuration	24
4.1.26 Parameter DpgaEnableBipolarDetector	25
4.1.27 Parameter DpgaLowDetectLimit	25
4.1.28 Parameter DpgaLowDetectFilterDuration	26
4.1.29 Parameter DpgaHighDetectLimit	26

4.1.30	Parameter DpgaHighDetectFilterDuration	26
4.1.31	Parameter DpgaCallback	27
4.1.32	Container DpgaBlankingTimeTriggerList	27
4.1.33	Parameter DpgaBTTriggerState	28
4.1.34	Container CommonPublishedInformation	28
4.1.35	Parameter ArReleaseMajorVersion	28
4.1.36	Parameter ArReleaseMinorVersion	29
4.1.37	Parameter ArReleaseRevisionVersion	29
4.1.38	Parameter ModuleId	30
4.1.39	Parameter SwMajorVersion	30
4.1.40	Parameter SwMinorVersion	30
4.1.41	Parameter SwPatchVersion	31
4.1.42	Parameter VendorApiInfix	31
4.1.43	Parameter VendorId	32
4.2	DPGA_IP	32
4.2.1	Module Dpga_Ip	33
5	Module Index	34
5.1	Software Specification	34
6	Module Documentation	35
6.1	Dpga	35
6.1.1	Detailed Description	35
6.1.2	Data Structure Documentation	36
6.1.2.1	struct Dpga_ConfigType	36
6.1.3	Macro Definition Documentation	36
6.1.3.1	DPGA_INIT_ID	36
6.1.3.2	DPGA_DEINIT_ID	36
6.1.3.3	DPGA_CONFIGURE_AMPLIFIER_ID	36
6.1.3.4	DPGA_VOLTAGE_MONITOR_SELF_TEST_ID	36
6.1.3.5	DPGA_AMPLIFIER_SELF_TEST_ID	36
6.1.4	Types Reference	37
6.1.4.1	Dpga_AmplifierConfigType	37
6.1.5	Function Reference	37
6.1.5.1	Dpga_Init()	37
6.1.5.2	Dpga_DeInit()	37
6.1.5.3	Dpga_ConfigureAmplifier()	37
6.2	Dpga_ip	39
6.2.1	Detailed Description	39
6.2.2	Data Structure Documentation	39
6.2.2.1	struct Dpga_Ip_AmplifierConfigType	39
6.2.2.2	struct Dpga_Ip_ConfigType	40
6.2.3	Types Reference	40
6.2.3.1	Dpga_Ip_CallbackType	40
6.2.4	Enum Reference	40
6.2.4.1	Dpga_Ip_StatusType	40
6.2.4.2	Dpga_Ip_EventType	40
6.2.5	Function Reference	41
6.2.5.1	Dpga_Ip_Init()	41
6.2.5.2	Dpga_Ip_DeInit()	41
6.2.5.3	Dpga_Ip_ConfigureAmplifier()	41
6.2.5.4	Dpga_Ip_IRQHandler()	42



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor DPGA for S32M24x. Dpga driver configuration parameters and deviations from the specification are described in Driver chapter of this document. Dpga driver requirements and APIs are described in the Dpga driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".

- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CDD	Complex Device Driver
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
DPGA	Differential Programmable Gain Amplifier
SIU	Systems Integration Unit
SWS	Software Specification
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	S32K1xx Reference Manual	Rev.14.1, 01/2024
2	S32M24x Reference Manual	Rev. 5, 12/2024
3	S32K1xx Data Sheet	Rev. 14, 08/2021
4	S32M2xx Data Sheet	Rev. 7, 12/2024
5	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
6	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
7	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
8	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
9	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
10	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
11	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
12	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
13	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)
- [Usage in non AUTOSAR context](#)

3.1 Requirements

Dpga is a Complex Device Driver (CDD), so there are no AUTOSAR requirements regarding this module. It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

The DPGA amplifies the voltage of the input signal, so that it can be measured with higher (compared to the non-amplified signal) resolution by an ADC.

3.3 Hardware Resources

The Dpga driver uses the DPGA hardware IP. For more details and availability please check the device reference manual.

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR CDD_Dpga Driver software specification in some places. The table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, not available, not testable or out of scope for the Dpga Driver.

Term	Definition
N/S	Out of scope

Term	Definition
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, not available, not testable or out of scope for the driver.

3.5 Driver Limitations

Dpga driver limitations:

- Blanking time was not tested

3.6 Driver usage and configuration tips

3.6.1 Dpga introduction

- In order to use the Dpga driver, the logical channel which maps a hardware unit must be first initialized using [Dpga_Init\(\)](#) function.
- Once a channel is initialized, it cannot be initialized again until it is de-initialized using [Dpga_DeInit\(\)](#) function.
- Dpga module amplifies the voltage of the input signal, so that it can be measured with higher (compared to the non-amplified signal) resolution by an ADC.
- In order to use Dpga module, it should be activated in McuResetConfig by checking DPGA Reset checkbox (deasserts hard reset input port of the module).
- Dpga has the following features:
 - Programmable amplification by 8, 16, 24, 32, 40, 50, 65, or 80
 - Both unipolar and bipolar input voltage ranges
 - Configurable input/output common voltage
 - Blanking time, used to protect the amplifier against saturation (support added, but has not been validated)
 - Voltage monitoring (not yet implemented in the driver)
 - Functional self-test for the amplifier and the voltage monitoring (not yet implemented)

3.6.2 Dpga Amplifier

- In order to allow the highest possible resolution and accuracy in the measurement of the input signal, you have to carefully configure the amplifier.

- Adjust the common voltage of the differential signal on the input so that the voltage on the plus and minus inputs of the amplifier cannot become negative.
- Adapt the gain of the amplifier so that in case the maximum voltage on the differential input in your application is amplified, the output of the amplifier is below its maximal output voltage.
- Compensate the offset of your amplifier for the chosen gain so that the unwanted voltage shift in the output signal of the amplifier is minimized. Best is that you measure the offset while there is 0 differential input voltage.
- Shift the common voltage of the output so that the dynamic range of the output signal is within the voltage range of the amplifier.
- The usual method to measure the Dpga output voltage is to use an on-chip ADC. However, if the ADC is not used, one can use a multimeter to measure the voltage on the AMPOUT pin. Before measuring it, one should enable (set to 1) the Dpga Out bit in IO FuncMux register in the Application Extension Controller.

3.6.3 Dpga Blanking Time

- Depending on the gain of the amplifier, that you set in your application, and on the dynamic range of the differential input, the transistor in the amplifier could saturate. In the saturation, the reaction of the output on any change on the input is slower than usual.
- One should avoid the saturation in order to maximize the usability of the DPGA. The blanking time can help you to avoid the saturation by disconnecting the input from the amplifier during the time when high voltages can occur on the differential input, e.g. due to ringing caused by switching on the input signal.
- The inputs of the amplifier are connected to ground while the blanking time is active.
- There are 6 blanking time trigger signals. You can select which event, so a low to high or a high to low transition on any of the 6 trigger signals, can start the blanking time. Once a trigger occurs, it always starts the full blanking time defined by the number of clock cycles, no matter whether the blanking time was already active when the trigger occurred.
- During the blanking time, the differential inputs to the DPGA are disconnected from the amplifier itself.

3.7 Runtime errors

The driver generates the following DET errors at runtime.

Function	Error Code	Condition triggering the error
Dpga_Init	DPGA_E_ALREADY_INITIALIZED	Calling Dpga_Init twice in a row
Dpga_Init	DPGA_E_INIT_FAILED	Dpga module is not initialized successfully
Dpga_Init	DPGA_E_PARAM_CONFIG	API is called with invalid configuration parameter

Function	Error Code	Condition triggering the error
Dpga_DeInit	DPGA_E_NOT_INITIALIZED	Dpga_Init() was not called before
Dpga_DeInit	DPGA_E_PARAM_CONFIG	API is called with invalid configuration parameter
Dpga_VoltageMonitorSelfTest	DPGA_E_NOT_INITIALIZED	Dpga_Init() was not called before
Dpga_VoltageMonitorSelfTest	DPGA_E_PARAM_CONFIG	API is called with invalid configuration parameter
Dpga_AmplifierSelfTest	DPGA_E_NOT_INITIALIZED	Dpga_Init() was not called before
Dpga_AmplifierSelfTest	DPGA_E_PARAM_CONFIG	API is called with invalid configuration parameter
Dpga_ConfigureAmplifier	DPGA_E_NOT_INITIALIZED	Dpga_Init() was not called before
Dpga_ConfigureAmplifier	DPGA_E_INVALID_POINTER	The AmplifierConfig pointer is NULL_P↵L_PTR
Dpga_ConfigureAmplifier	DPGA_E_PARAM_CONFIG	API is called with invalid configuration parameter
Dpga_GetVersionInfo	DPGA_E_INVALID_POINTER	The VersionInfo pointer is NULL_P↵TR

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: `#define <Mip>Conf_<Container_ShortName>_<Container_ID>`

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

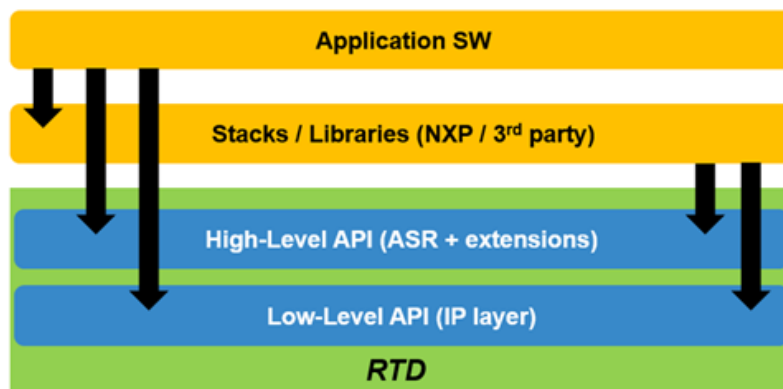


Figure 3.1 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
 1. Initialization of the device clock tree so other periphery can be initialized
 2. Configuration of the pinout so peripherals with external connections can communicate with the outside world
 3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
 4. Initialization of specific drivers
 5. Runtime functionality
 6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

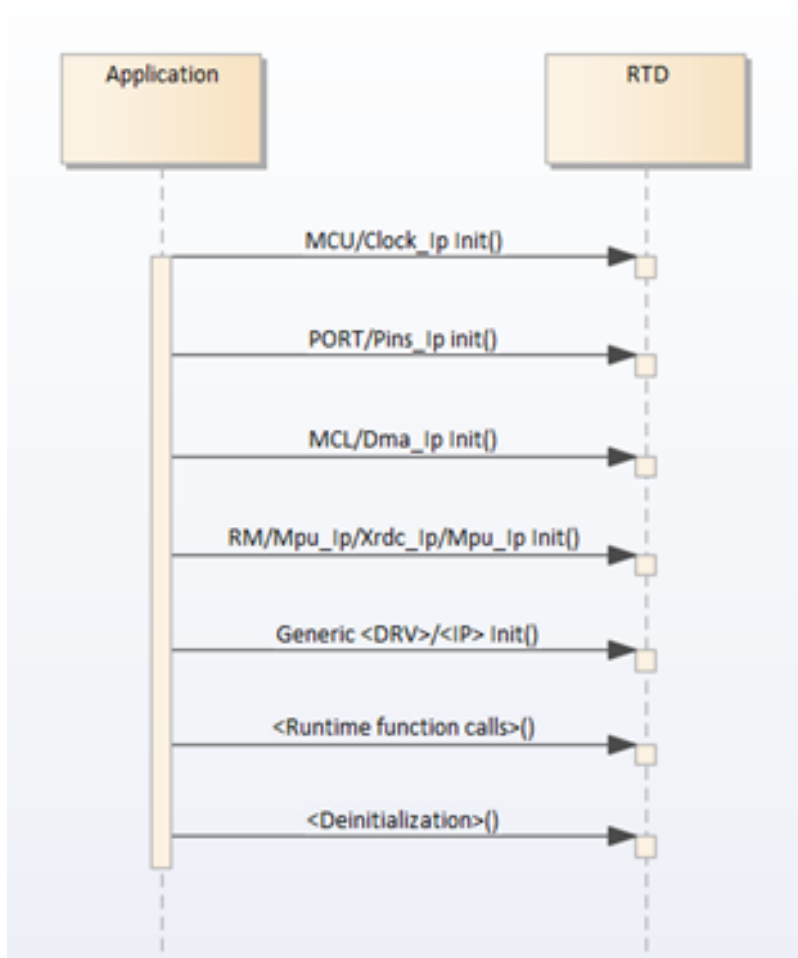


Figure 3.2 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**.xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

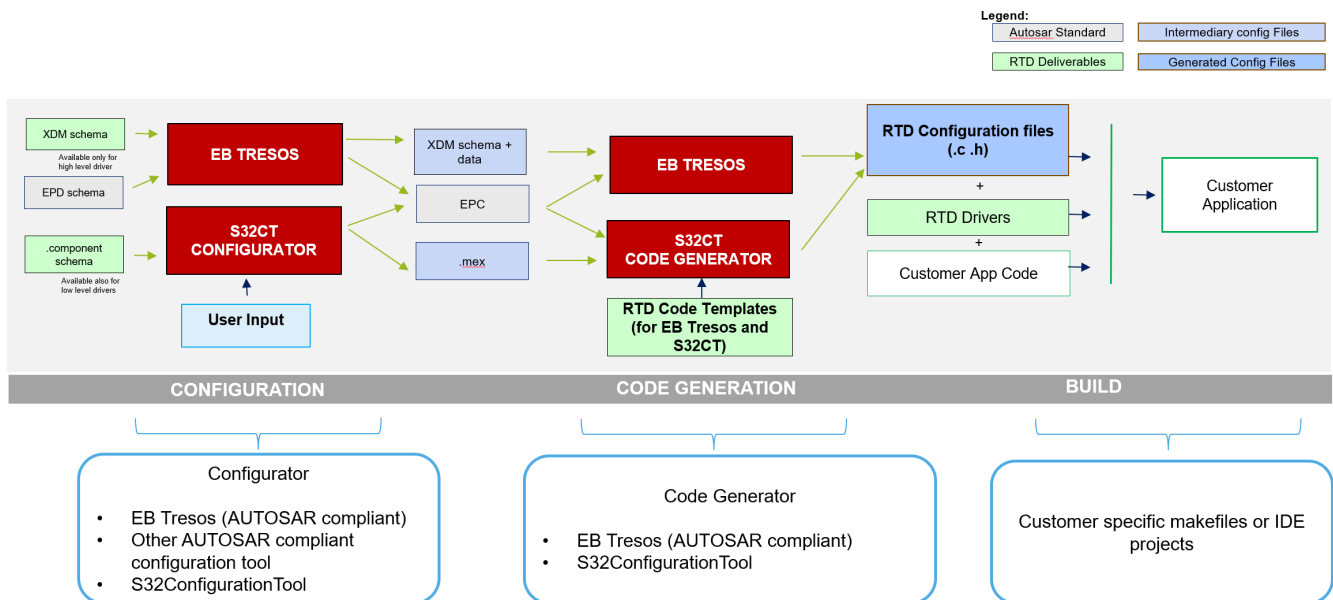


Figure 4.1 Configuration generation flow

- [Dpga](#)
- [DPGA_IP](#)

4.1 Dpga

- Module [Dpga](#)
 - Container [GeneralConfiguration](#)
 - * Parameter [DpgaEnableVoltageMonitoring](#)
 - * Parameter [DpgaDevErrorDetect](#)
 - * Parameter [DpgaIpDevErrorDetect](#)
 - * Parameter [DpgaMultiPartitionSupport](#)
 - * Parameter [DpgaEnableUserModeSupport](#)
 - * Parameter [DpgaVersionInfoApi](#)
 - * Parameter [DpgaVoltageMonitorSelfTestApi](#)
 - * Parameter [DpgaAmplifierSelfTestApi](#)
 - * Parameter [DpgaAmplifierSelfTestTolerance](#)
 - * Parameter [DpgaTimeoutMethod](#)
 - * Parameter [DpgaTimeoutValue](#)
 - * Reference [DpgaEcucPartitionRef](#)
 - * Reference [DpgaAmplifierSelfTestAdcReference](#)
 - Container [DpgaChannel](#)
 - * Parameter [DpgaChannelId](#)
 - * Parameter [DpgaHwChannel](#)
 - * Container [DpgaConfiguration](#)
 - Parameter [DpgaAmplifierGain](#)
 - Parameter [DpgaOutCommonModeVoltage](#)
 - Parameter [DpgaOffset](#)
 - Parameter [DpgaInCommonModeCoarse](#)
 - Parameter [DpgaInCommonModeFine](#)
 - Parameter [DpgaBlankingTimeDuration](#)
 - Parameter [DpgaEnableBipolarDetector](#)
 - Parameter [DpgaLowDetectLimit](#)
 - Parameter [DpgaLowDetectFilterDuration](#)
 - Parameter [DpgaHighDetectLimit](#)
 - Parameter [DpgaHighDetectFilterDuration](#)
 - Parameter [DpgaCallback](#)
 - Container [DpgaBlankingTimeTriggerList](#)
 - Parameter [DpgaBTTriggerState](#)
 - Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)

Configuration Parameters

- * Parameter [ArReleaseMinorVersion](#)
- * Parameter [ArReleaseRevisionVersion](#)
- * Parameter [ModuleId](#)
- * Parameter [SwMajorVersion](#)
- * Parameter [SwMinorVersion](#)
- * Parameter [SwPatchVersion](#)
- * Parameter [VendorApiInfix](#)
- * Parameter [VendorId](#)

4.1.1 Module Dpga

Configuration of the Differential Programmable Gain Amplifier (DPGA) module.

Included containers:

- [GeneralConfiguration](#)
- [DpgaChannel](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.1.2 Container GeneralConfiguration

This container contains the global configuration parameters of the Non-Autosar Dpga driver.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Parameter DpgaEnableVoltageMonitoring

Enable/Disable Voltage Monitoring

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.4 Parameter DpgaDevErrorDetect

Switches the Development Error Detection and Notification ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.5 Parameter DpgaIpDevErrorDetect

Enables/disables development error detection for Dpga Ip.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.6 Parameter DpgaMultiPartitionSupport

This parameter globally enables the possibility to support multi-partition. If this parameter is enabled, at least one EcucPartition needs to be defined (in all variants).

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.7 Parameter DpgaEnableUserModeSupport

No special measures need to be taken to run Dpga module from user mode.

The Dpga driver code can be executed at any time from both supervisor and user mode.

(Please refer to chapter "5.8 User Mode support" in the IM file)

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.8 Parameter DpgaVersionInfoApi

Vendor specific: Adds / removes the service Dpga_GetVersionInfo() from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.9 Parameter DpgaVoltageMonitorSelfTestApi

Vendor specific: Adds / removes the voltage monitor self-test from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.10 Parameter DpgaAmplifierSelfTestApi

Vendor specific: Adds / removes the amplifier self-test from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.11 Parameter DpgaAmplifierSelfTestTolerance

Vendor specific: This tolerance is a deviation between the target value and the actual value in ADC measurement that can be accepted.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	100
max	255
min	0

4.1.12 Parameter DpgaTimeoutMethod

Configures the timeout method for Dpga

Based on this selection a certain timeout method from OsIf will be used in the driver.

Note: If OSIF_COUNTER_SYSTEM or OSIF_COUNTER_CUSTOM are selected make sure the corresponding timer is enabled in OsIf General configuration.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COUNTER_CUSTOM']

4.1.13 Parameter DpgaTimeoutValue

This is a timeout (microseconds) value which is used to wait for each synchronization transfer

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	65535
max	4294967295
min	1

4.1.14 Reference DpgaEcucPartitionRef

Maps the Dpga driver to zero or multiple ECUC partitions to make the driver

API available in the according partition.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.15 Reference DpgaAmplifierSelfTestAdcReference

Vendor specific: Reference to the ADC configuration.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Configuration Parameters

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Adc/AdcConfigSet/AdcHwUnit

4.1.16 Container DpgaChannel

This container contains the configuration (parameters) of the Dpga Controller(s).

Included subcontainers:

- [DpgaConfiguration](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.17 Parameter DpgaChannelId

Identifies the Dpga channel

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	1
min	0

4.1.18 Parameter DpgaHwChannel

Selects hardware channel.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	DPGA_0
literals	['DPGA_0']

4.1.19 Container DpgaConfiguration

This container contains the configuration (parameters) of the Dpga Controller(s).

Included subcontainers:

- [DpgaBlankingTimeTriggerList](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.20 Parameter DpgaAmplifierGain

Vendor specific: Select the gain of the differential amplifier.

GAIN_8: Amplify by 8

GAIN_16: Amplify by 16

GAIN_24: Amplify by 24

GAIN_32: Amplify by 32

GAIN_40: Amplify by 40

GAIN_50: Amplify by 50

GAIN_65: Amplify by 65

GAIN_80: Amplify by 80

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP

Configuration Parameters

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	GAIN_8
literals	['GAIN_8', 'GAIN_16', 'GAIN_24', 'GAIN_32', 'GAIN_40', 'GAIN_50', 'GAIN_65', 'GAIN_80']

4.1.21 Parameter DpgaOutCommonModeVoltage

Vendor specific: Configure the common mode voltage of the output of the amplifier.

VREF_DIV_12: Reference voltage divided by 12

VREF_DIV_6: Reference voltage divided by 6

VREF_DIV_4: Reference voltage divided by 4

VREF_DIV_2: Reference voltage divided by 2

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	VREF_DIV_2
literals	['VREF_DIV_12', 'VREF_DIV_6', 'VREF_DIV_4', 'VREF_DIV_2']

4.1.22 Parameter DpgaOffset

Vendor specific: Configure the value to compensate the offset of the differential amplifier.

0: No offset

1: +3mV

2: +6mV

7: +21mV

8: No offset

9: -3mV

10: -6mV

15: -21mV

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.1.23 Parameter DpgaInCommonModeCoarse

Vendor specific: Configure the common mode of the differential inputs of the amplifier.

NO_SHIFT: No shift

SHIFT_200: Shift with 200 uA current out of both inputs

SHIFT_100: Shift with 100 uA current out of both inputs

SHIFT_50: Shift with 50 uA current out of both inputs

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	SHIFT_0
literals	['SHIFT_0', 'SHIFT_200', 'SHIFT_100', 'SHIFT_50']

4.1.24 Parameter DpgaInCommonModeFine

Vendor specific: Adjust the common mode of the differential inputs of the amplifier by changing the current out of one of the differential inputs.

*0: Input Common Mode Coarse * 1.0000 out of minus input*

*1: Input Common Mode Coarse * 1.0025 out of minus input*

*2: Input Common Mode Coarse * 1.0050 out of minus input*

*31: Input Common Mode Coarse * 1.0775 out of minus input*

*32: Input Common Mode Coarse * 1.0000 out of plus input*

*33: Input Common Mode Coarse * 1.0025 out of plus input*

*34: Input Common Mode Coarse * 1.0050 out of plus input*

*63: Input Common Mode Coarse * 1.0775 out of plus input*

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0
max	63
min	0

4.1.25 Parameter DpgaBlankingTimeDuration

Vendor specific: Blanking Time duration, in number of module clock cycles.

The actual number of clock cycles is this number + 1.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true

Property	Value
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.1.26 Parameter DpgaEnableBipolarDetector

Vendor specific: Enable the low detector. Use the low detector for bipolar measurements.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.27 Parameter DpgaLowDetectLimit

Vendor specific: When the output of the amplifier is lower than the limit defined here, a low detect event is generated.

The low detect limit is actually $(1 + \text{this value}) / 64$ of the supply voltage.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.1.28 Parameter DpgaLowDetectFilterDuration

Vendor specific: The minimal time duration that the low detect must be activer before a low detect output flag is set.

This value + 1 is the actual minimal time duration in number of module clock cycles.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.1.29 Parameter DpgaHighDetectLimit

Vendor specific: When the output of the amplifier is higher than the limit defined here, a high detect event is generated.

The high detect limit is actually $(48 + \text{this value}) / 64$ of the supply voltage.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.1.30 Parameter DpgaHighDetectFilterDuration

Vendor specific: The minimal time duration that the high detect must be activer before a high detect output flag is set.

This value + 1 is the actual minimal time duration in number of module clock cycles.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0
max	255
min	0

4.1.31 Parameter DpgaCallback

Dpga callback. This function will be called for all Dpga events.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	NULL_PTR

4.1.32 Container DpgaBlankingTimeTriggerList

Vendor specific: List of the Blanking Time triggers

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	6
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.1.33 Parameter DpgaBTTriggerState

Vendor specific: Select the state of the blanking time trigger that enables the blanking time counter.

NO_EDGE: This trigger cannot start the BT counter

FALLING_EDGE: The falling edge of this trigger starts the BT counter from 0

RISING_EDGE: The rising edge of this trigger starts the BT counter from 0

BOTH_EDGES: Both edges of this trigger start the BT counter from 0

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NO_EDGE
literals	['NO_EDGE', 'FALLING_EDGE', 'RISING_EDGE', 'BOTH_EDGES']

4.1.34 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.35 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.1.36 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	7
max	7
min	7

4.1.37 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0

Configuration Parameters

Property	Value
max	0
min	0

4.1.38 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	255
max	255
min	255

4.1.39 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	3
max	3
min	3

4.1.40 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.41 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.42 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_<VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.1.43 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

4.2 DPGA_IP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Dpga_Ip](#)
 - Container [GeneralConfiguration](#)
 - * Parameter [DpgaIpDevErrorDetect](#)
 - * Parameter [DpgaVersionInfoApi](#)
 - * Parameter [DpgaTimeoutMethod](#)
 - * Parameter [DpgaTimeoutValue](#)
 - * Parameter [DpgaEnableUserModeSupport](#)
 - * Parameter [DpgaMultiPartitionSupport](#)

- * Parameter [DpgaEnableVoltageMonitoring](#)
- * Parameter [DpgaVoltageMonitorSelfTestApi](#)
- * Parameter [DpgaAmplifierSelfTestApi](#)
- * Parameter [DpgaAmplifierSelfTestTolerance](#)
- * Reference [DpgaEcucPartitionRef](#)
- * Reference [DpgaAmplifierSelfTestAdcReference](#)
- Container [DpgaChannel](#)

4.2.1 Module Dpga_Ip

Dpga_ip config set

Included containers:

- [GeneralConfiguration](#)
- [DpgaChannel](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Dpga	35
Dpga_ip	39

Chapter 6

Module Documentation

6.1 Dpga

6.1.1 Detailed Description

Data Structures

- struct [Dpga_ConfigType](#)

Dpga module configuration structure This structure is used to provide the configuration parameters for the Dpga module. [More...](#)

Macros

- `#define` [DPGA_INIT_ID](#)
APIs service IDs.
- `#define` [DPGA_DEINIT_ID](#)
- `#define` [DPGA_CONFIGURE_AMPLIFIER_ID](#)
- `#define` [DPGA_VOLTAGE_MONITOR_SELF_TEST_ID](#)
- `#define` [DPGA_AMPLIFIER_SELF_TEST_ID](#)

Types Reference

- typedef [Dpga_Ip_AmplifierConfigType](#) [Dpga_AmplifierConfigType](#)

Differential Amplifier configuration structure This structure is used to provide the configuration parameters for the Differential Amplifier.

Function Reference

- Std_ReturnType [Dpga_Init](#) (const [Dpga_ConfigType](#) *const Config)
Initialize the DPGA module.
- Std_ReturnType [Dpga_DeInit](#) (void)
Deinitialize the DPGA module.
- Std_ReturnType [Dpga_ConfigureAmplifier](#) (const [Dpga_AmplifierConfigType](#) *AmplifierConfig)
Reload Amplifier configuration.

6.1.2 Data Structure Documentation

6.1.2.1 struct Dpga_ConfigType

Dpga module configuration structure This structure is used to provide the configuration parameters for the Dpga module.

Definition at line 132 of file CDD_Dpga_Types.h.

6.1.3 Macro Definition Documentation

6.1.3.1 DPGA_INIT_ID

```
#define DPGA_INIT_ID
```

APIs service IDs.

Service IDs for the DPGA driver APIs. [Dpga_Init\(\)](#) ID

Definition at line 140 of file CDD_Dpga.h.

6.1.3.2 DPGA_DEINIT_ID

```
#define DPGA_DEINIT_ID
```

[Dpga_Deinit\(\)](#) ID

Definition at line 141 of file CDD_Dpga.h.

6.1.3.3 DPGA_CONFIGURE_AMPLIFIER_ID

```
#define DPGA_CONFIGURE_AMPLIFIER_ID
```

[Dpga_ConfigureAmplifier\(\)](#) ID

Definition at line 142 of file CDD_Dpga.h.

6.1.3.4 DPGA_VOLTAGE_MONITOR_SELF_TEST_ID

```
#define DPGA_VOLTAGE_MONITOR_SELF_TEST_ID
```

[Dpga_VoltageMonitorSelfTest\(\)](#) ID

Definition at line 143 of file CDD_Dpga.h.

6.1.3.5 DPGA_AMPLIFIER_SELF_TEST_ID

```
#define DPGA_AMPLIFIER_SELF_TEST_ID
```

[Dpga_AmplifierSelfTest\(\)](#) ID

Definition at line 144 of file CDD_Dpga.h.

6.1.4 Types Reference

6.1.4.1 Dpga_AmplifierConfigType

```
typedef Dpga_Ip_AmplifierConfigType Dpga_AmplifierConfigType
```

Differential Amplifier configuration structure This structure is used to provide the configuration parameters for the Differential Amplifier.

Definition at line 125 of file CDD_Dpga_Types.h.

6.1.5 Function Reference

6.1.5.1 Dpga_Init()

```
Std_ReturnType Dpga_Init (
    const Dpga_ConfigType *const Config )
```

Initialize the DPGA module.

This function initializes the DPGA module:

- Maps the logical channel to the hardware channel
- Initializes the channel

Parameters

in	<i>Config</i>	Pointer to the configuration structure
----	---------------	--

6.1.5.2 Dpga_DeInit()

```
Std_ReturnType Dpga_DeInit (
    void )
```

Deinitialize the DPGA module.

This function deinitializes the DPGA module to the reset values. The driver must be initialized before calling [Dpga_DeInit\(\)](#).

6.1.5.3 Dpga_ConfigureAmplifier()

```
Std_ReturnType Dpga_ConfigureAmplifier (
    const Dpga_AmplifierConfigType * AmplifierConfig )
```

Reload Amplifier configuration.

This function is used to configure the Amplifier at runtime. The user can store specific configuration values after device power up and reload them after each device reset.

Parameters

in	<i>AmplifierConfig</i>	Pointer to the Amplifier configuration structure
----	------------------------	--

Module Documentation

Returns

Std_ReturnType

6.2 Dpga_ip

6.2.1 Detailed Description

Data Structures

- struct [Dpga_Ip_AmplifierConfigType](#)
Differential Amplifier configuration structure. [More...](#)
- struct [Dpga_Ip_ConfigType](#)
Dpga Ip module configuration structure. [More...](#)

Types Reference

- typedef void(* [Dpga_Ip_CallbackType](#)) ([Dpga_Ip_EventType](#) Event)
Callback used for DPGA module notifications. Possible notifications: low/high voltage detect, parity check.

Enum Reference

- enum [Dpga_Ip_StatusType](#)
Dpga Ip Status.
- enum [Dpga_Ip_EventType](#)
Events which can trigger DPGA notification callback.

Function Reference

- [Dpga_Ip_StatusType Dpga_Ip_Init](#) (const uint8 Instance, const [Dpga_Ip_ConfigType](#) *const Config)
Initialize the DPGA module.
- [Dpga_Ip_StatusType Dpga_Ip_DeInit](#) (const uint8 Instance)
Deinitialize the DPGA module.
- [Dpga_Ip_StatusType Dpga_Ip_ConfigureAmplifier](#) (const uint8 Instance, const [Dpga_Ip_AmplifierConfigType](#) *AmplifierConfig)
Reload Amplifier configuration.
- [Dpga_Ip_StatusType Dpga_Ip_IRQHandler](#) (const uint8 Instance)
Handle DPGA interrupt.

6.2.2 Data Structure Documentation

6.2.2.1 struct [Dpga_Ip_AmplifierConfigType](#)

Differential Amplifier configuration structure.

This structure is used to provide the configuration parameters for the Differential Amplifier

Definition at line 138 of file [Dpga_Ip_Types.h](#).

Data Fields

uint8	AmplifierGain	The gain of the Amplifier
uint8	OutCommonModeVoltage	Common Mode Voltage of the output of the Amplifier
uint8	AmplifierOffset	Amplifier offset

Data Fields

uint8	InCommonModeCoarse	The common mode of the differential inputs of the Amplifier
uint8	InCommonModeFine	Adjust the common mode of the differential inputs of the amplifier

6.2.2.2 struct Dpga_Ip_ConfigType

Dpga Ip module configuration structure.

This structure is used to provide the configuration parameters for the Dpga Ip module.

Definition at line 153 of file Dpga_Ip_Types.h.

Data Fields

Dpga_Ip_AmplifierConfigType *	AmplifierConfig	Pointer to the Amplifier Config structure
uint8	BlankingTimeDuration	Blanking time duration, in number of module cycles
uint8	BTTriggerStateArray[DPGA_IP_BT_STATE] NUMBER OF TRIGGERS	State of the triggers Triggers 0 - 5: on which edge to start the counter for each trigger
Dpga_Ip_CallbackType	DpgaCallback	Dpga callback function. This will be called for all Dpga events.

6.2.3 Types Reference

6.2.3.1 Dpga_Ip_CallbackType

```
typedef void(* Dpga_Ip_CallbackType) (Dpga\_Ip\_EventType Event)
```

Callback used for DPGA module notifications. Possible notifications: low/high voltage detect, parity check.

Definition at line 130 of file Dpga_Ip_Types.h.

6.2.4 Enum Reference

6.2.4.1 Dpga_Ip_StatusType

```
enum Dpga\_Ip\_StatusType
```

Dpga Ip Status.

Definition at line 98 of file Dpga_Ip_Types.h.

6.2.4.2 Dpga_Ip_EventType

```
enum Dpga\_Ip\_EventType
```

Events which can trigger DPGA notification callback.

Definition at line 111 of file Dpga_Ip_Types.h.

6.2.5 Function Reference

6.2.5.1 Dpga_Ip_Init()

```
Dpga_Ip_StatusType Dpga_Ip_Init (
    const uint8 Instance,
    const Dpga_Ip_ConfigType *const Config )
```

Initialize the DPGA module.

This function initializes the DPGA module.

Parameters

in	<i>Instance</i>	DPGA instance
in	<i>Config</i>	Pointer to the configuration structure

6.2.5.2 Dpga_Ip_DeInit()

```
Dpga_Ip_StatusType Dpga_Ip_DeInit (
    const uint8 Instance )
```

Deinitialize the DPGA module.

This function deinitializes the DPGA module. The driver can't be used until initialized again.

Parameters

in	<i>Instance</i>	DPGA instance
----	-----------------	---------------

6.2.5.3 Dpga_Ip_ConfigureAmplifier()

```
Dpga_Ip_StatusType Dpga_Ip_ConfigureAmplifier (
    const uint8 Instance,
    const Dpga_Ip_AmplifierConfigType * AmplifierConfig )
```

Reload Amplifier configuration.

This function is used to configure the Amplifier at runtime. The user can store specific configuration values after device power up and reload them after each device reset.

Parameters

in	<i>Instance</i>	DPGA instance
in	<i>AmplifierConfig</i>	Pointer to the Amplifier configuration structure

6.2.5.4 Dpga_Ip_IRQHandler()

```
Dpga_Ip_StatusType Dpga_Ip_IRQHandler (
    const uint8 Instance )
```

Handle DPGA interrupt.

This function handles DPGA interrupt events.

Parameters

in	<i>Instance</i>	DPGA instance
----	-----------------	---------------

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.